
Transacciones y Control de Concurrencia en Bases de Datos

Introducción

Las bases de datos modernas se enfrentan constantemente al desafío de gestionar operaciones simultáneas mientras mantienen la integridad de los datos. En este apunte, vamos a explorar dos conceptos fundamentales que resuelven este desafío: las transacciones y el control de concurrencia.

Imagina un banco donde miles de clientes realizan operaciones sobre sus cuentas simultáneamente. ¿Cómo aseguramos que si una persona transfiere dinero a otra, el monto se reste de una cuenta y se sume a la otra de manera coherente, incluso si el sistema falla a mitad del proceso? Las transacciones son la respuesta a este problema.

¿Qué es una Transacción?

Una transacción es una **secuencia de operaciones** que se ejecutan como una **unidad lógica de trabajo indivisible**. Podemos entenderla como un "paquete" de operaciones que deben **completarse** en su **totalidad** o **no ejecutarse** en absoluto.

Por ejemplo, consideremos una transferencia bancaria:

1. Verificar que la cuenta origen tenga fondos suficientes
2. Restar el monto de la cuenta origen
3. Sumar el monto a la cuenta destino
4. Registrar la operación en el historial de transacciones

Si cualquiera de estos pasos falla, **toda la transferencia debe cancelarse** para mantener la **consistencia** del sistema.

Propiedades ACID

Las transacciones se rigen por cuatro propiedades esenciales conocidas como ACID:

Atomicidad (Atomicity)

La atomicidad garantiza que todas las operaciones dentro de una transacción se traten como una unidad indivisible. Esto significa que o bien todas las operaciones se ejecutan correctamente, o ninguna se aplica. No hay estados intermedios.

Usando la analogía del desarrollo de software: si un equipo está construyendo una aplicación y un desarrollador añade código mientras otro lo elimina, es como si no hubiera ocurrido ningún cambio. Pero si todos añaden código sin eliminar nada, la base de código crece de manera integral.

Consistencia (Consistency)

La consistencia asegura que una transacción sólo puede llevar la base de datos de un estado válido a otro estado igualmente válido, respetando todas las reglas, restricciones e integridad referencial definidas.

Por ejemplo, si una tabla tiene la restricción de que una columna solo puede contener valores enteros, la consistencia garantiza que ninguna transacción pueda insertar un valor no entero en esa columna.

Aislamiento (Isolation)

El aislamiento determina cómo se manejan las transacciones concurrentes y cómo los cambios realizados por una transacción son visibles para otras transacciones ejecutándose simultáneamente.

MySQL soporta cuatro niveles de aislamiento, en orden de menor a mayor restricción:

1. **READ UNCOMMITTED**: Permite leer datos no confirmados por otras transacciones, lo que puede causar lecturas sucias (valores temporales que luego son revertidos).
2. **READ COMMITTED**: Solo permite leer datos confirmados, evitando las lecturas sucias. Sin embargo, puede producir lecturas no repetibles (cuando una transacción lee el mismo dato dos veces y obtiene valores diferentes).
3. **REPEATABLE READ**: Garantiza que si una transacción lee un dato, seguirá viendo el mismo valor aunque otras transacciones lo modifiquen y confirmen.

Previene lecturas no repetibles, pero puede provocar lecturas fantasma (cuando aparecen filas nuevas entre dos lecturas).

4. **SERIALIZABLE**: El nivel más estricto, hace que las transacciones se comporten como si se ejecutaran en serie, una después de otra, eliminando todos los problemas de concurrencia posibles, pero reduciendo significativamente el rendimiento.

MySQL utiliza por defecto el nivel **READ UNCOMMITTED**, aunque puede cambiarse según las necesidades específicas.

Durabilidad (Durability)

La durabilidad garantiza que una vez que una transacción ha sido confirmada, sus cambios permanecen en la base de datos incluso ante fallos del sistema, como cortes de energía o errores de hardware.

MySQL logra la durabilidad mediante el registro anticipado (write-ahead logging), escribiendo primero un registro de la transacción en disco antes de modificar los datos reales. Si el sistema falla, estos registros permiten reconstruir el estado más reciente de la base de datos.

Control de Concurrencia y Bloqueos

El control de concurrencia es el mecanismo que permite que múltiples usuarios accedan y modifiquen los datos simultáneamente sin crear inconsistencias.

Tipos de Bloqueos en MySQL

MySQL utiliza varios tipos de bloqueos para controlar el acceso concurrente a los datos:

1. **Bloqueos compartidos (Shared locks)**: Permiten a múltiples transacciones leer los mismos datos simultáneamente, pero impiden que escriban sobre ellos.
2. **Bloqueos exclusivos (Exclusive locks)**: Impiden que otras transacciones lean o escriban los datos bloqueados.
3. **Bloqueos de intención (Intent locks)**: Indican que una transacción planea adquirir un bloqueo más específico sobre ciertos datos.
4. **Bloqueos a nivel de fila (Row-level locks)**: Permiten bloquear filas individuales en lugar de tablas completas, mejorando el rendimiento en entornos con alta concurrencia.

Control de Concurrencia Multiversión (MVCC)

MySQL utiliza un mecanismo llamado Control de Concurrencia Multiversión (MVCC) que permite a múltiples transacciones trabajar con los mismos datos sin interferirse entre sí.

MVCC funciona manteniendo diferentes versiones de los datos. Cuando una transacción inicia, captura una instantánea del estado actual de la base de datos y trabaja con esa versión. Esto permite que otras transacciones sigan operando con su propia versión de los datos, evitando conflictos.

Para optimizar la concurrencia, es recomendable:

- Mantener las transacciones lo más cortas posible
- Evitar transacciones de larga duración que mantengan bloqueos por períodos extensos
- Acceder a las tablas en el mismo orden en todas las transacciones para evitar interbloqueos

Implementación de Transacciones en MySQL

Comandos Básicos

MySQL ofrece varias sentencias para trabajar con transacciones:

1. **START TRANSACTION / BEGIN:** Inicia una nueva transacción.
2. **COMMIT:** Confirma todos los cambios realizados durante la transacción.
3. **ROLLBACK:** Deshace todos los cambios realizados durante la transacción.
4. **SET autocommit:** Controla si las operaciones se confirman automáticamente o requieren un COMMIT explícito.

Ejemplo Práctico

Veamos un ejemplo de transferencia bancaria utilizando transacciones:

Dentro de un procedimiento almacenado:

```
START TRANSACTION;
-- verificamos si el saldo es suficiente
SELECT @saldo := balance FROM accounts WHERE user_id = 1;

IF @saldo >= 1000 THEN
    -- restamos de la cuenta origen
    UPDATE accounts SET balance = balance - 1000 WHERE user_id = 1;

    -- sumamos a la cuenta destino
    UPDATE accounts SET balance = balance + 1000 WHERE user_id = 2;

    -- registramos la transferencia
    INSERT INTO transactions (from_id, to_id, amount, date)
    VALUES (1, 2, 1000, NOW());

    COMMIT;
ELSE
    ROLLBACK;
END IF;
```

Manejo de Errores en Transacciones

Es importante prever y manejar adecuadamente los errores que pueden ocurrir durante una transacción.

Uso de Manejadores de Excepciones

MySQL permite definir manejadores para capturar excepciones SQL y responder apropiadamente:

```
START TRANSACTION;
DECLARE EXIT HANDLER FOR SQLEXCEPTION
BEGIN
    ROLLBACK;
    SELECT 'Error detectado' AS mensaje;
END;

-- código de la transacción

COMMIT;
```

NOTA: esto debe usarse dentro de un stored procedure

Puntos de Guardado (Savepoints)

Los puntos de guardado permiten definir marcadores dentro de una transacción para poder realizar rollbacks parciales:

```
START TRANSACTION;
-- primera operación
UPDATE accounts SET balance = balance - 1000 WHERE user_id = 1;

-- definir un punto de guardado
SAVEPOINT transferencia_iniciada;

-- Segunda operación
UPDATE accounts SET balance = balance + 1000 WHERE user_id = 2;

-- si algo sale mal, podemos volver al punto de guardado
-- ROLLBACK TO SAVEPOINT transferencia_iniciada;

-- si todo está bien, confirmamos
COMMIT;
```

Escenarios de Concurrencia y Problemas Comunes

Problemas de Concurrencia

Cuando múltiples transacciones operan simultáneamente, pueden surgir varios problemas:

1. **Lecturas sucias (Dirty reads):** Una transacción lee datos modificados por otra transacción que aún no ha sido confirmada.

2. **Lecturas no repetibles (Non-repeatable reads):** Una transacción lee el mismo registro dos veces y obtiene valores diferentes porque otra transacción ha modificado los datos entre las lecturas.
3. **Lecturas fantasma (Phantom reads):** Una transacción ejecuta la misma consulta dos veces y la segunda vez aparecen filas adicionales que cumplen los criterios de búsqueda porque otra transacción ha insertado nuevos registros.
4. **Actualizaciones perdidas (Lost updates):** Dos transacciones leen el mismo registro, lo modifican y lo escriben, causando que la última actualización sobrescriba la primera.
5. **Interbloqueo (Deadlock):** Dos transacciones se bloquean mutuamente porque cada una espera recursos que la otra tiene bloqueados. MySQL detecta automáticamente los interbloqueos y resuelve la situación abortando una de las transacciones.

Estrategias para Evitar Problemas de Concurrency

1. **Seleccionar el nivel de aislamiento adecuado** según los requisitos de la aplicación:
 - Para operaciones de solo lectura menos críticas: READ COMMITTED
 - Para la mayoría de las aplicaciones: REPEATABLE READ (el predeterminado de MySQL)
 - Para aplicaciones que requieren consistencia absoluta: SERIALIZABLE
2. **Optimizar la estructura de las transacciones:**
 - Mantener las transacciones lo más cortas posible
 - Minimizar el número de operaciones dentro de una transacción
 - Evitar operaciones de usuario (como esperar entrada) dentro de una transacción
3. **Utilizar bloqueos explícitos** cuando sea necesario:

```
-- bloqueo para lectura (compartido)
SELECT * FROM table_name WHERE id = 10 LOCK IN SHARE MODE;

-- bloqueo para escritura (exclusivo)
SELECT * FROM table_name WHERE id = 10 FOR UPDATE;
```

4. **Acceder a las tablas en un orden consistente** en todas las transacciones para evitar interbloqueos.

Conclusión

Las transacciones y el control de concurrencia son fundamentales para mantener la integridad y consistencia de los datos en sistemas que permiten múltiples usuarios simultáneos.

Al comprender los principios ACID, los niveles de aislamiento, los mecanismos de bloqueo y las estrategias para manejar errores, los desarrolladores pueden diseñar sistemas robustos que protejan los datos incluso en entornos de alta concurrencia o ante fallos del sistema.

MySQL, especialmente con su motor InnoDB, ofrece un sólido conjunto de herramientas para implementar transacciones y control de concurrencia de manera efectiva. El desafío está en encontrar el equilibrio adecuado entre consistencia y rendimiento según los requisitos específicos de cada aplicación.

Recursos

<https://www.freecodecamp.org/news/how-to-use-mysql-transactions/> - How to Use MySQL Transactions, Free code camp.

<https://dev.mysql.com/doc/refman/8.0/en/sql-transactional-statements.html> - Documentación de MySQL